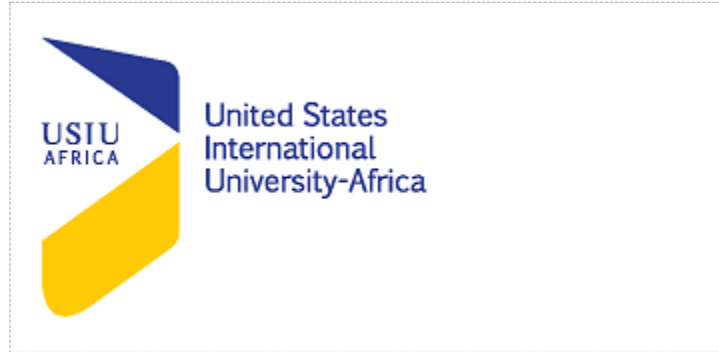


UNITED STATES INTERNATIONAL UNIVERSITY – AFRICA



SCHOOL OF SCIENCE AND TECHNOLOGY

SMART HEALTH SYMPTOM CHECKER AND DOCTOR RECOMMENDATION SYSTEM

**Midterm Project Report
SWE3090XA**

Submitted by:

Amy Wanjugu Njau

Registration Number: 669008

Supervisor: Mr Fredrick Ogore

**Submission Date: June 15th 2026
Summer Semester 2026**

Declaration

I, Amy Njau, hereby declare that this midterm project report titled "Smart Health Symptom Checker and Doctor Recommendation System" is my own original work and has not been submitted, in whole or in part, for any award or examination at this or any other university or institution of higher learning.

All sources of information and materials used in the preparation of this report have been duly acknowledged through appropriate citation and referencing. I confirm that this work adheres to the academic integrity and honesty policies of the United States International University Africa (USIU-Africa), and that any assistance received has been properly credited.

Acknowledgement

I wish to express my sincere gratitude to all those who supported me throughout the first half of this project.

First, I extend my deepest appreciation to my project supervisor for their invaluable guidance, constructive feedback, and continued encouragement during the requirements, design, and early development phases of this work. Their direction has been instrumental in shaping both the scope and the technical approach of the system.

I am equally grateful to the faculty and staff of the Department of Computing at USIU-Africa for providing the academic foundation, resources, and learning environment that made this project possible.

My appreciation also goes to my classmates and peers, whose code reviews, discussions, and honest critique helped refine the system's design and improve the quality of the prototype. Finally, I thank my family and friends for their unwavering moral support, patience, and motivation throughout this journey.

Dedication

This work is dedicated to everyone who has ever hesitated before seeking medical help unsure whether their symptoms were serious enough, or where to turn for care.

And to my family, whose belief in the power of education and technology to improve lives continues to inspire me.

Abstract

Access to timely and affordable medical guidance remains a significant challenge in many developing regions, where the doctor-to-patient ratio is critically low and patients frequently delay seeking care due to cost, distance, or uncertainty about the severity of their symptoms. This project presents the design and development of the Smart Health Symptom Checker and Doctor Recommendation System, a cross-platform mobile application that empowers users to make informed, early health decisions.

The primary objective of the system is threefold: to accept user-reported symptoms and return a ranked list of probable medical conditions; to recommend the appropriate type of medical specialist for the predicted condition; and to identify nearby qualified doctors or healthcare facilities using the user's real-time location. The diagnostic logic is implemented through a rule-based inference engine that maps symptom combinations to candidate conditions using a weighted symptom–disease knowledge base, deliberately positioning the application as a guidance and triage tool rather than a clinical diagnostic device.

The project follows an Agile development methodology organised into iterative sprints across a ten-week schedule. At this midterm stage, the requirements gathering, literature review, system architecture, and core data models are complete, and a functional prototype of the rule-based diagnostic engine together with the initial mobile user-interface screens has been implemented and tested. Comparative analysis of existing symptom checkers confirmed a clear gap in solutions that combine localised, location-aware doctor recommendation with an explainable, lightweight diagnostic approach suited to low-connectivity environments.

The remaining phases will focus on completing the geolocation-based recommendation module, full backend integration, end-to-end testing, and user acceptance testing. The anticipated contribution of this work is an accessible, intuitive digital-health tool that shortens the path between symptom onset and appropriate care, particularly for underserved communities.

Keywords: symptom checker, rule-based inference, digital health, mHealth, doctor recommendation, geolocation, triage, Flutter, healthcare accessibility.

Acronyms and Definitions

The following acronyms, abbreviations, and terms are used throughout this report.

Term / Acronym	Definition
API	Application Programming Interface a defined set of rules that allows software components to communicate.
mHealth	Mobile Health the practice of medicine and public health supported by mobile devices.
UI / UX	User Interface / User Experience the visual design and overall experience of interacting with the application.
DFD	Data Flow Diagram a graphical representation of how data moves through a system.
ERD	Entity Relationship Diagram a model of the data entities in a system and the relationships between them.
REST	Representational State Transfer an architectural style for designing networked APIs.
JSON	JavaScript Object Notation a lightweight, human-readable data-interchange format.
MVP	Minimum Viable Product the smallest functional version of a product that delivers core value.
EHR	Electronic Health Record a digital version of a patient's medical history.
SDK	Software Development Kit a collection of tools for building applications on a specific platform.
CRUD	Create, Read, Update, Delete the four basic operations of persistent data storage.
WHO	World Health Organization the United Nations agency responsible for international public health.
UAT	User Acceptance Testing final testing performed by end users to confirm the system meets requirements.
Rule-based engine	A reasoning component that applies predefined IF–THEN rules (here, a weighted symptom matrix) to infer probable conditions.

Table of Contents

Contents

Declaration.....	2
Acknowledgement.....	3
Dedication.....	4
Abstract.....	5
Acronyms and Definitions	6
Table of Contents.....	7
Chapter One: Introduction	9
1.1 Background of the Study	9
1.2 Emerging Issues.....	9
1.3 Problem Statement.....	10
1.4 Objectives.....	10
1.4.1 Main Objective	10
1.4.2 Specific Objectives	10
1.5 Justification and Significance.....	10
Chapter Two: Literature Review	12
2.1 Review of Existing Systems.....	12
2.1.1 WebMD Symptom Checker	12
2.1.2 Ada Health	12
2.1.3 Babylon Health.....	12
2.1.4 Buoy Health and K Health	12
2.1.5 Academic Evaluations	12
2.2 Comparative Analysis	12
2.3 Identified Gaps	13
2.4 Unique Contributions	13
Chapter Three: Methodology.....	14
3.1 Development Approach	14
3.1.1 Project Timeline	14
3.2 Resources and Tools.....	15
3.2.1 Hardware	15
3.2.2 Software, Frameworks, and Services	15
3.3 Requirements Gathering.....	15
3.3.1 Functional Requirements	15
3.3.2 Non-Functional Requirements.....	16

Chapter Four: Implementation.....	17
4.1 System Analysis and Design	17
4.2 Context Diagram.....	18
4.3 Data Flow Diagram (DFD)	18
4.4 Entity Relationship (ER) Diagram	19
Chapter Five: Coding	21
5.1 Code Snippets with Titles	21
5.1.1 Symptom–Disease Knowledge Base.....	21
5.1.2 Rule-Based Diagnostic Engine (Weighted Scoring).....	21
5.1.3 Diagnosis API Endpoint.....	22
5.1.4 Nearby Provider Lookup (Geolocation)	23
5.1.5 Flutter Symptom-Input Screen (excerpt).....	23
5.2 Screenshots of Interfaces and Outputs.....	23
5.3 Explanation of Key Modules and Algorithms.....	26
5.3.1 Knowledge Base Module.....	26
5.3.2 Diagnostic Engine and Scoring Algorithm	27
5.3.3 Specialist Recommendation Module	27
5.3.4 Geolocation and Provider Module	27
5.3.5 Safety and Disclaimer Handling	27
Chapter Six: Conclusions and Recommendations.....	28
6.1 Summary of Findings.....	28
6.2 Recommendations for Future Work	28
6.3 References	28
6.4 Appendices.....	29

Right-click and select “Update Field” in Word to populate page numbers.

Chapter One: Introduction

1.1 Background of the Study

Access to timely and accurate medical guidance remains a major challenge across many parts of the world, and particularly in developing regions where the ratio of doctors to patients is critically low. According to the World Health Organization, large populations in low- and middle-income countries continue to face shortages of healthcare workers, long travel distances to facilities, and high out-of-pocket consultation costs. As a result, many individuals delay seeking professional medical attention, not because they are unwilling, but because they are uncertain about the severity of their symptoms or do not know which type of practitioner to consult.

At the same time, mobile-phone penetration in these regions has grown rapidly, placing internet-connected devices in the hands of populations that were previously difficult to reach through conventional healthcare infrastructure. This convergence creates a meaningful opportunity for mobile health (mHealth) solutions to bridge part of the gap between symptom onset and appropriate care.

The Smart Health Symptom Checker and Doctor Recommendation System respond to this opportunity by providing a digital-first solution that empowers users to make informed health decisions. By inputting their symptoms into a mobile application, users receive evidence-based condition suggestions generated through a rule-based analysis engine. The system then advises them on the appropriate type of medical specialist to consult and goes a step further by recommending nearby qualified doctors or healthcare facilities based on the user's real-time location. In this way, the application acts as an accessible first point of guidance not a replacement for professional care, but a bridge toward it.

1.2 Emerging Issues

Several emerging issues motivate and shape this project:

- **Rising demand for self-directed care:** Patients increasingly turn to online sources to interpret their symptoms before deciding whether and where to seek help. Unstructured web searches, however, often produce alarming or contradictory information.
- **Proliferation of digital symptom checkers:** A growing number of commercial symptom-checking applications exist, but many are designed for high-income contexts, require constant connectivity, or do not connect the user to local, reachable care.
- **Localisation gap:** Few existing tools account for the realities of developing regions nearby facilities, locally available specialists, and low-bandwidth conditions.
- **Trust, safety, and explainability:** As automated health tools become more common, there is increased scrutiny on how recommendations are generated. Rule-based systems offer transparency and explainability that opaque models cannot easily match, which is valuable in a sensitive domain such as health.

- Data privacy expectations: Handling even limited health-related data raises legitimate concerns around consent, storage, and confidentiality that any responsible solution must address.

1.3 Problem Statement

Many individuals lack access to timely, affordable, and location-aware medical guidance, leading to delayed diagnoses and preventable deterioration of their health. Existing solutions are either too generic to be trustworthy, too expensive for the average user, or not localised to the user's context failing to connect them to qualified care that is actually within reach.

There is therefore a need for an accessible mobile application that can interpret a user's symptoms, suggest probable conditions in an explainable manner, recommend the appropriate type of specialist, and direct the user to nearby qualified healthcare providers all through a simple interface usable by people with varying levels of digital literacy.

1.4 Objectives

1.4.1 Main Objective

To design and develop a cross-platform mobile application that accepts user-reported symptoms, returns a ranked list of probable medical conditions, recommends an appropriate medical specialist, and identifies nearby qualified healthcare providers based on the user's real-time location.

1.4.2 Specific Objectives

1. To design and develop a mobile application that accepts user-inputted symptoms and returns a ranked list of possible medical conditions.
2. To implement a rule-based diagnostic engine capable of mapping symptom combinations to probable diseases with reasonable accuracy.
3. To integrate a specialist recommendation module that suggests the appropriate type of doctor based on the predicted condition.
4. To incorporate location-based services that identify and recommend nearby qualified healthcare providers or medical facilities.
5. To provide a simple, intuitive user interface accessible to users with varying levels of digital literacy.

1.5 Justification and Significance

This project is justified by the persistent and widening gap between the demand for healthcare guidance and the supply of accessible professional care in developing regions. By combining symptom analysis with localised, location-aware doctor recommendation, the system addresses a need that existing tools only partially serve.

The significance of the work can be summarised across several dimensions:

- **For users:** It provides a free, easy-to-use first point of guidance that reduces uncertainty, encourages earlier care-seeking, and helps users reach the right kind of practitioner faster.
- **For the healthcare system:** By guiding patients toward the appropriate specialist and nearby facilities, the tool can help reduce mis-directed visits and ease pressure on overstretched general services.
- **For underserved communities:** The emphasis on localisation and a lightweight, explainable approach makes the solution suitable for low-connectivity environments often overlooked by commercial alternatives.
- **Academically:** The project demonstrates the practical application of software-engineering principles requirements engineering, system design, rule-based inference, API integration, and Agile delivery to a real-world problem of social value.

Chapter Two: Literature Review

This chapter reviews existing symptom-checker and digital-health systems, compares their capabilities against the goals of this project, identifies the gaps they leave open, and states the unique contributions of the proposed system.

2.1 Review of Existing Systems

Several digital symptom-checking and health-guidance solutions exist in the market and in academic literature. The most relevant are reviewed below.

2.1.1 WebMD Symptom Checker

WebMD provides a widely used web-based symptom checker that allows users to select symptoms on a body map and returns a list of possible conditions with accompanying articles. It is information-rich but is primarily an educational reference; it does not connect users to specific local practitioners and is oriented toward a North American audience and healthcare context.

2.1.2 Ada Health

Ada is a mobile symptom-assessment application that conducts a structured, conversational interview and returns possible causes together with guidance on next steps. It is well designed and clinically informed, but it is built primarily for connected, higher-income markets, does not focus on surfacing nearby local providers, and its underlying assessment logic is largely opaque to the end user.

2.1.3 Babylon Health

Babylon combined symptom checking with telemedicine consultations, allowing users to speak with clinicians remotely. While powerful, it depends on reliable connectivity, an available clinician network, and a subscription or payment model, which limits its accessibility in low-resource settings.

2.1.4 Buoy Health and K Health

Buoy Health and K Health both use data-driven approaches to triage symptoms and suggest next steps, with K Health additionally drawing on large clinical datasets. These tools illustrate the value of structured triage but, like the others, are tailored to specific national healthcare systems and do not provide localised facility recommendation for developing-region users.

2.1.5 Academic Evaluations

Independent academic audits of symptom checkers notably the BMJ audit study by Semigran and colleagues have found that the diagnostic and triage accuracy of these tools varies considerably. This body of work underscores two design principles adopted by this project: that such tools should be positioned explicitly as guidance rather than diagnosis, and that an explainable, well-validated rule base is preferable to an opaque one in a safety-sensitive domain.

2.2 Comparative Analysis

The table below compares representative existing systems against the core capabilities targeted by this project.

System	Symptom Analysis	Specialist Recommendation	Location-Aware Provider Recommendation	Explainable Logic	Suited to Low-Connectivity Context
WebMD Symptom Checker	Yes	Partial	No	Partial	No
Ada Health	Yes	Partial	No	No	No
Babylon Health	Yes	Yes (via clinician)	Partial	No	No
Buoy / K Health	Yes	Partial	No	No	No
Proposed System	Yes	Yes	Yes	Yes	Yes

Table 2.1: Comparative analysis of existing symptom-checker systems.

2.3 Identified Gaps

The review reveals several consistent gaps in existing solutions:

- **Lack of localisation:** Most tools are designed for specific high-income healthcare systems and do not surface nearby, reachable providers relevant to a developing-region user.
- **Limited specialist routing:** Few systems explicitly translate a probable condition into a clear recommendation of which type of specialist to consult.
- **Opacity of reasoning:** Many modern tools provide little visibility into how a recommendation was produced, which undermines user trust in a sensitive domain.
- **Connectivity and cost barriers:** Solutions that depend on continuous connectivity, large models, or paid consultations are poorly suited to low-resource environments.

2.4 Unique Contributions

The Smart Health Symptom Checker and Doctor Recommendation System address these gaps through the following contributions:

- **Integrated, end-to-end guidance:** a single workflow that moves the user from symptom entry to probable condition, to the right specialist, to a nearby provider.
- **Location-aware recommendation:** use of real-time geolocation to recommend qualified doctors and facilities that are within reach of the user.
- **Explainable rule-based inference:** a transparent weighted symptom–disease matrix whose reasoning can be surfaced to the user, increasing trust and safety.
- **Lightweight, accessible design:** an interface and architecture deliberately suited to varying digital-literacy levels and lower-connectivity conditions.

Chapter Three: Methodology

This chapter describes the development approach adopted for the project, the resources and tools used, and the functional and non-functional requirements gathered during the analysis phase.

3.1 Development Approach

The project follows an Agile software-development methodology, implemented through short, iterative sprints. Agile was selected over a strictly sequential approach such as Waterfall because the requirements of a health-guidance tool benefit from continuous testing, frequent feedback, and incremental refinement particularly for the diagnostic rule base, which must be validated and tuned progressively rather than fixed up front.

Development is organised into four key phases, each spanning one or more sprints:

6. **Requirements and Research:** review existing symptom-checker solutions, define the symptom–disease rule base, and finalise the system architecture.
7. **System Design:** design UI/UX wireframes, data models, the API structure, and the rule-based inference engine.
8. **Development and Integration:** build the mobile application, implement the diagnostic engine, integrate geolocation services, and connect the backend.
9. **Testing and Deployment:** conduct unit, integration, and user-acceptance testing, then deploy to a staging environment for final review.

At each sprint boundary, working increments are reviewed against the milestones defined in the project timeline (Section 3.2.4), and the rule base and interface are refined based on testing outcomes and peer feedback.

3.1.1 Project Timeline

The high-level schedule spans approximately ten weeks. The shaded rows below indicate work completed as of this midterm report.

Week	Milestone / Activity	Deliverable	Status
1 – 2	Requirements gathering, literature review, system design	Requirements document, wireframes	Completed
3 – 4	Database design, backend setup, diagnostic rule-engine development	Working rule engine (prototype)	Completed
5 – 6	Mobile UI development (symptom input, results screen)	UI prototype / alpha build	In progress
7 – 8	Geolocation integration, specialist recommendation module	Beta build with location feature	Pending
9	End-to-end testing, bug fixes, user-acceptance testing	Tested application	Pending

Week	Milestone / Activity	Deliverable	Status
10	Final documentation, repository cleanup, project submission	Final report and repository	Pending

Table 3.1: Project timeline and current status.

3.2 Resources and Tools

The following hardware, software, frameworks, and services support the development of the system.

3.2.1 Hardware

- Personal laptop/computer for development.
- Android device (and emulator) for testing the mobile application.

3.2.2 Software, Frameworks, and Services

Category	Tool / Technology	Purpose
Frontend	Flutter (Dart)	Cross-platform mobile UI development (Android first, iOS-ready).
Backend	Node.js with Express (REST API)	Serves the diagnostic engine and recommendation endpoints.
Database	Firebase Firestore	Stores the symptom–disease knowledge base, specialist mappings, and provider data.
Location Services	Google Maps Platform API / Geolocation API	Detects user location and finds nearby providers.
Diagnostic Logic	Rule-based inference engine	Weighted symptom–disease matrix / IF–THEN decision logic.
Version Control	Git and GitHub	Source-code management and collaboration.
IDE / Tooling	VS Code, Android Studio	Development, debugging, and emulation.

Table 3.2: Tools and technologies.

3.3 Requirements Gathering

Requirements were elicited through a review of existing symptom-checker systems, study of publicly available medical knowledge bases and WHO guidance, and consideration of the constraints of the target users and context. They are grouped into functional and non-functional requirements.

3.3.1 Functional Requirements

The system shall:

10. Allow users to input symptoms through structured selection and/or free-text entry.

11. Analyse the entered symptoms using the rule-based engine and return a ranked list of probable conditions.
12. Recommend the appropriate type of medical specialist for the top predicted condition(s).
13. Detect the user's real-time location (with permission) and recommend nearby qualified doctors or facilities.
14. Display a clear disclaimer that the application provides guidance only and is not a substitute for professional medical advice.
15. Handle location-permission denial gracefully and continue to provide symptom and specialist guidance.

3.3.2 Non-Functional Requirements

16. **Usability:** the interface must be simple and intuitive for users with varying levels of digital literacy.
17. **Performance:** symptom analysis should return results within a few seconds on a typical mobile connection.
18. **Reliability:** the diagnostic engine should produce consistent results for the same input and degrade gracefully when services are unavailable.
19. **Security and Privacy:** user-entered health data and location must be handled with appropriate consent, minimisation, and confidentiality.
20. **Portability:** the architecture should target Android first while allowing future iOS extension without a full rewrite.
21. **Maintainability:** the rule base should be structured so that new symptoms, conditions, and specialists can be added without changing core logic.

Chapter Four: Implementation

This chapter presents the system analysis and design completed during the first half of the project, including the overall architecture and the standard analysis and design models: the context diagram, data flow diagram, and entity relationship diagram.

4.1 System Analysis and Design

The system adopts a three-tier architecture comprising a presentation layer, an application/logic layer, and a data layer.

- **Presentation layer:** the Flutter mobile application, responsible for symptom input, displaying ranked results and specialist advice, and showing nearby providers on a map/list.
- **Application layer:** the Node.js/Express backend, which exposes REST endpoints, hosts the rule-based diagnostic engine, performs specialist mapping, and orchestrates calls to the location service.
- **Data layer:** Firebase Firestore, which stores the symptom–disease knowledge base, condition-to-specialist mappings, provider records, and anonymised query logs; the Google Maps Platform supplies external place data.

The general flow of control is: the user enters symptoms in the app → the app sends the symptom set to the backend → the rule engine scores candidate conditions and returns a ranked list with recommended specialists → the app requests nearby providers for the chosen specialist using the user's location → results are displayed with a guidance disclaimer.

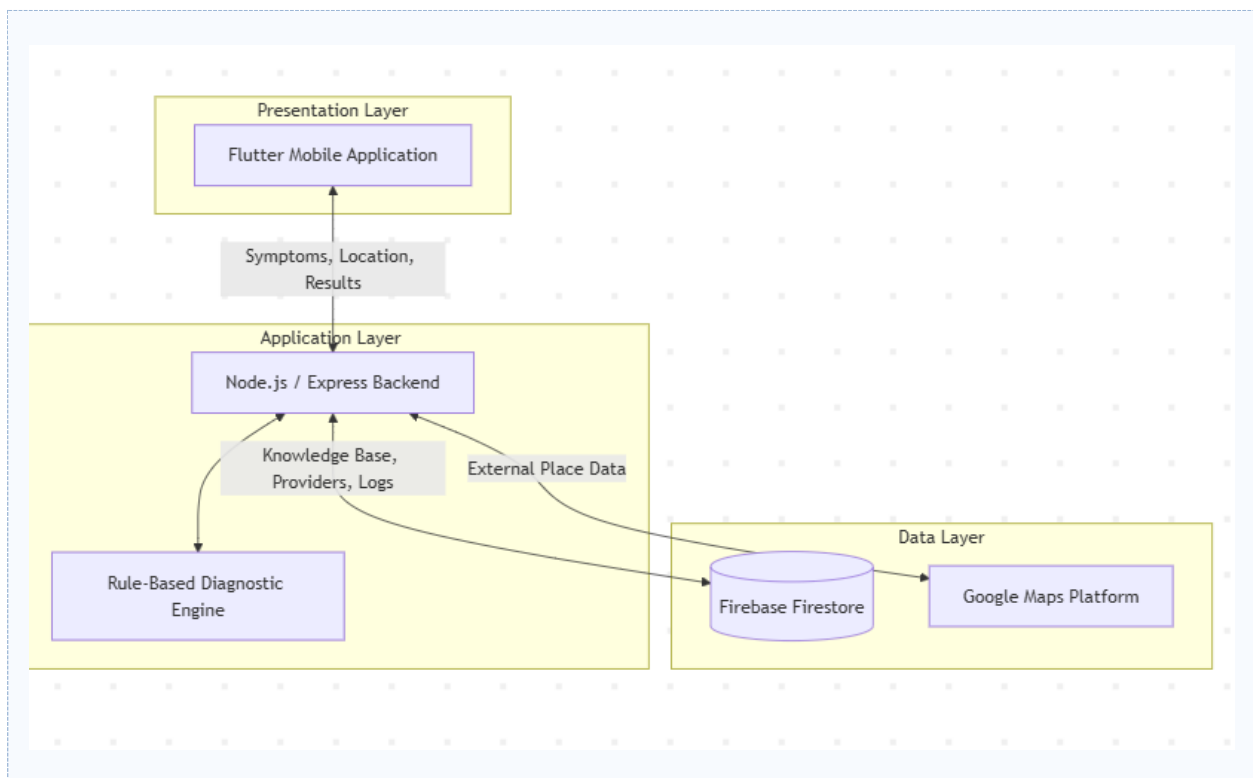


Figure 4.1: High-level system architecture.

4.2 Context Diagram

The context (Level-0 DFD) diagram represents the system as a single process interacting with its external entities. The principal external entities and data flows are summarised below and should be drawn as the formal context diagram.

- **User (Patient):** submits symptoms and location permission; receives ranked conditions, specialist advice, and nearby provider recommendations.
- **Google Maps Platform (external service):** receives location and specialty queries; returns nearby places and details.
- **Knowledge Base / Database:** provides symptom–disease rules and specialist mappings to the system.

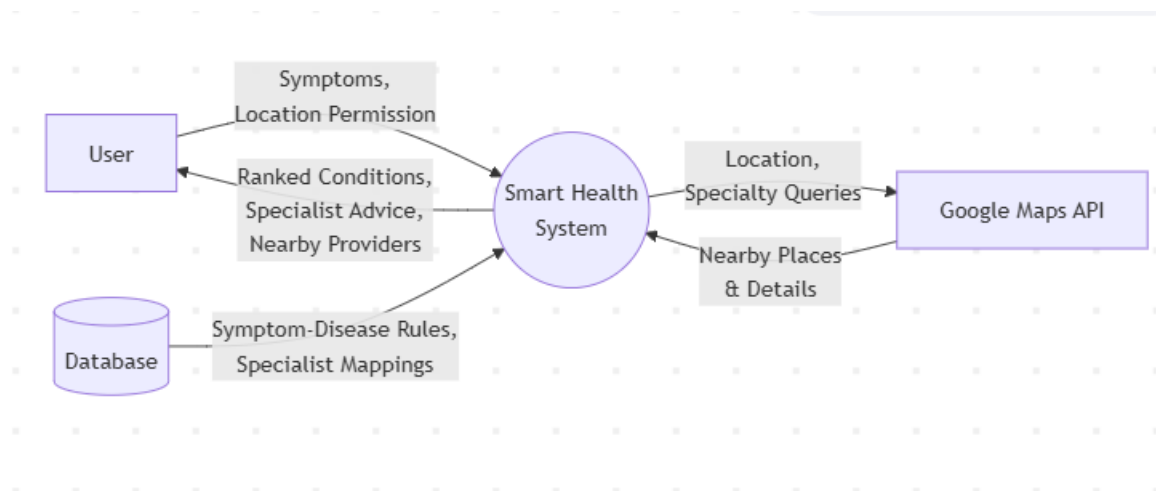


Figure 4.2: Context diagram (Level-0 DFD).

4.3 Data Flow Diagram (DFD)

The Level-1 DFD decomposes the system into its main internal processes. The core processes are:

22. **Capture Symptoms** — validates and structures the user's symptom input.
23. **Analyse Symptoms (Rule Engine)** — scores candidate conditions against the weighted symptom matrix and produces a ranked list.
24. **Recommend Specialist** — maps top condition(s) to the appropriate specialist type.
25. **Recommend Nearby Providers** — uses the user's location and specialist type to query and rank nearby facilities.
26. **Present Results** — compiles conditions, specialist advice, providers, and the disclaimer for display.

Data stores include the Symptom–Disease Knowledge Base, the Specialist Mapping store, the Provider store, and an anonymised Query Log.

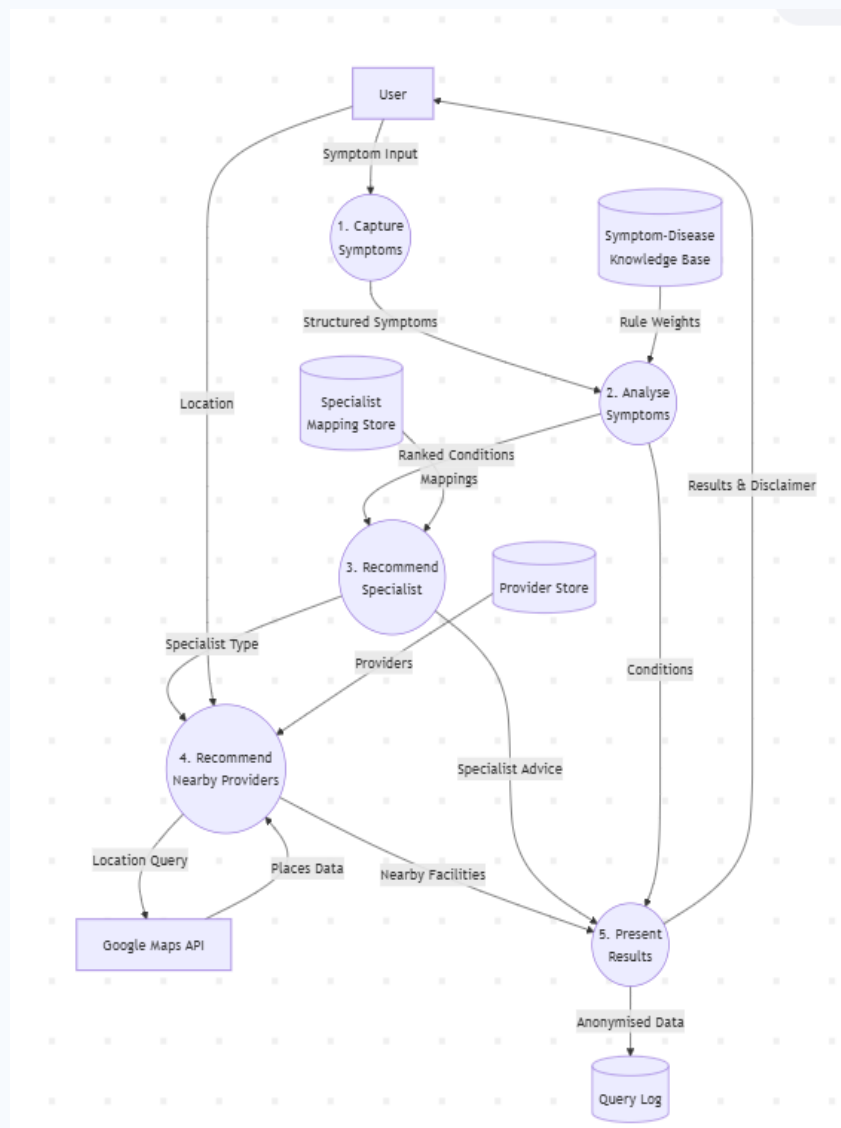


Figure 4.3: Level-1 data flow diagram.

4.4 Entity Relationship (ER) Diagram

The data model is built around the following principal entities and relationships. These should be rendered as the formal ER diagram.

Entity	Key Attributes	Relationships
User	user_id (PK), name, location, created_at	Initiates many Queries (1–M).
Symptom	symptom_id (PK), name, category	Linked to Conditions via Condition_Symptom (M–M).
Condition	condition_id (PK), name, description, severity	Linked to Symptoms (M–M); maps to one Specialist (M–1).

Entity	Key Attributes	Relationships
Condition_Symptom	condition_id (FK), symptom_id (FK), weight	Associative entity carrying the rule weight.
Specialist	specialist_id (PK), type, description	Recommended by many Conditions; matched to many Providers.
Provider	provider_id (PK), name, specialty, latitude, longitude, rating	Offers one or more Specialist types (M–M).
Query	query_id (PK), user_id (FK), symptoms, result, timestamp	Belongs to one User; produces one Result set.

Table 4.1: Core entities of the data model.

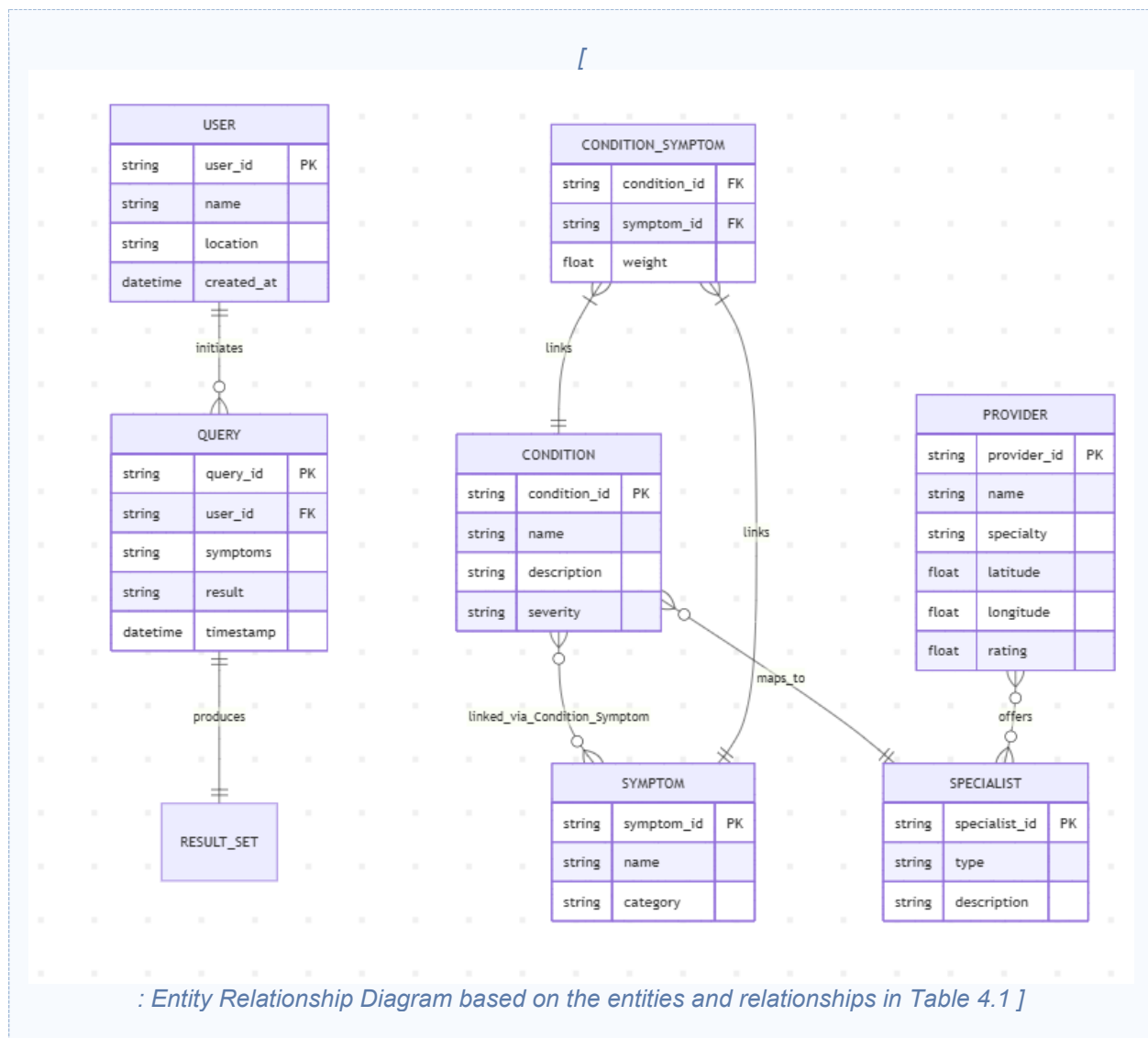


Figure 4.4: Entity relationship diagram.

Chapter Five: Coding

This chapter presents representative code from the prototype implemented during the first half of the project, screenshots of the current interface, and an explanation of the key modules and algorithms. The diagnostic engine and the initial mobile screens are functional; the geolocation and full backend integration are in progress.

5.1 Code Snippets with Titles

5.1.1 Symptom–Disease Knowledge Base

The knowledge base encodes each condition as a set of weighted symptoms. Weights represent how strongly a symptom indicates a condition, enabling an explainable, tunable rule base.

knowledge_base.json

```
1 {  
2   "conditions": [  
3     {  
4       "id": "malaria",  
5       "name": "Malaria",  
6       "specialist": "General Physician",  
7       "severity": "high",  
8       "symptoms": {  
9         "fever": 0.9, "chills": 0.8, "headache": 0.5,  
10        "fatigue": 0.4, "nausea": 0.4, "sweating": 0.6  
11      }  
12    },  
13    {  
14      "id": "common_cold",  
15      "name": "Common Cold",  
16      "specialist": "General Physician",  
17      "severity": "low",  
18      "symptoms": {  
19        "runny_nose": 0.9, "sore_throat": 0.7,  
20        "cough": 0.6, "sneezing": 0.7, "headache": 0.3  
21      }  
22    }  
23  ]  
24 }  
25
```

5.1.2 Rule-Based Diagnostic Engine (Weighted Scoring)

The engine computes a normalised match score for each condition by summing the weights of the symptoms the user reported and dividing by the maximum possible weight for that condition. Conditions are then ranked in descending order of score.

diagnosisEngine.js

```

CourseController.java | 17 Untitled-1.json | 15 // Score a single condition against the user's reported symptoms | 15 function s
1 // Score a single condition against the user's reported symptoms
2
3 Midterm Project Report — SWE3090XA
4
5 Amy Njau | Page 21
6 ✓ function scoreCondition(condition, userSymptoms) {
7   const weights = condition.symptoms;
8   const maxScore = Object.values(weights).reduce((a, b) => a + b, 0);
9   let matched = 0;
10  ✓ for (const symptom of userSymptoms) {
11    if (weights[symptom]) matched += weights[symptom];
12  }
13  return maxScore > 0 ? matched / maxScore : 0; // normalised 0..1
14 }
15 // Return a ranked list of probable conditions
16 ✓ function diagnose(userSymptoms, knowledgeBase, threshold = 0.2) {
17   return knowledgeBase.conditions
18   ✓ .map(c => ({
19     id: c.id,
20     name: c.name,
21     specialist: c.specialist,
22     severity: c.severity,
23     confidence: +(scoreCondition(c, userSymptoms) * 100).toFixed(1)
24   }))
25   .filter(r => r.confidence >= threshold * 100)
26   .sort((a, b) => b.confidence - a.confidence);
27 }
28 module.exports = { diagnose, scoreCondition };

```

5.1.3 Diagnosis API Endpoint

The Express endpoint accepts the symptom set, runs the engine, and returns the ranked conditions together with the recommended specialist for the top result.

routes/diagnose.js

```

1 const express = require(&#39;express&#39;);
2 const router = express.Router();
3 const { diagnose } = require(&#39;../services/diagnosisEngine&#39;);
4 const knowledgeBase = require(&#39;../data/knowledge_base.json&#39;);
5 router.post(&#39;/diagnose&#39;, (req, res) => {
6   const { symptoms } = req.body;
7   ✓ if (!Array.isArray(symptoms) || symptoms.length === 0) {
8     return res.status(400).json({ error: &#39;No symptoms provided.&#39; });
9   }
10  const results = diagnose(symptoms, knowledgeBase);
11  const recommendedSpecialist = results.length ? results[0].specialist : null;
12  ✓ return res.json({
13    results,
14    recommendedSpecialist,
15    disclaimer: &#39;This is a guidance tool, not a substitute for professional medical advice.&#39;
16  });
17 });
18 module.exports = router;

```

5.1.4 Nearby Provider Lookup (Geolocation)

Given the user's coordinates and the recommended specialist type, the service queries the Google Maps Platform for nearby providers and returns a ranked subset. This module is under active development.

services/providerService.js (in progress)

```
> Users > amnja > JS service.js > ...
1  const axios = require('axios');
2  const KEY = process.env.MAPS_API_KEY;
3  async function findNearbyProviders(lat, lng, specialist, radius = 5000) {
4    const keyword = encodeURIComponent(specialist + ' doctor clinic');
5    const url = `https://maps.googleapis.com/maps/api/place/nearbysearch/json` +
6    `?location=${lat},${lng}&radius=${radius}&keyword=${keyword}&key=${KEY}`;
7    const { data } = await axios.get(url);
8    return (data.results || [])
9    .map(p => ({
10     name: p.name,
11     address: p.vicinity,
12     rating: p.rating || null,
13     location: p.geometry.location
14   }));
15   .sort((a, b) => (b.rating || 0) - (a.rating || 0));
16   .slice(0, 10);
17 }
18 module.exports = { findNearbyProviders };
```

5.1.5 Flutter Symptom-Input Screen (excerpt)

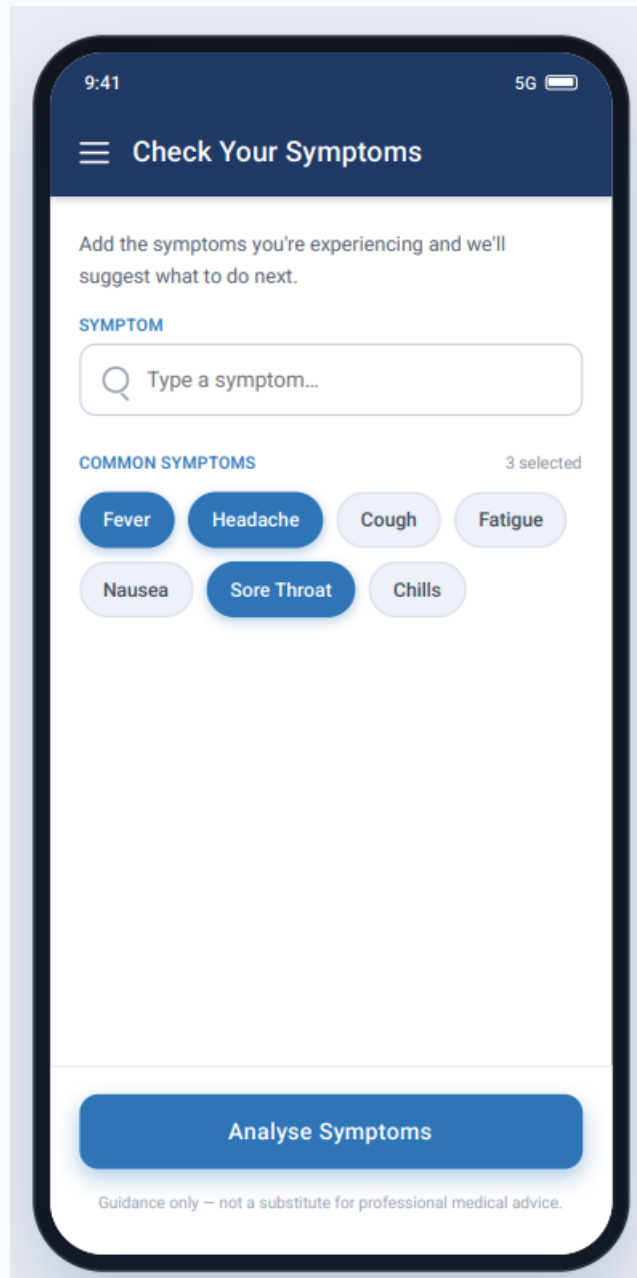
On the client, the user selects symptoms and submits them to the backend. The excerpt below shows the call that retrieves and displays results.

lib/services/api_service.dart

```
Future<DiagnosisResult> getDiagnosis(List<String> symptoms) async {
  final response = await http.post(
    Uri.parse('$baseUrl/api/diagnose'),
    headers: {'Content-Type': 'application/json'},
    body: jsonEncode({'symptoms': symptoms}),
  );
  if (response.statusCode == 200) {
    return DiagnosisResult.fromJson(jsonDecode(response.body));
  } else {
    throw Exception('Failed to retrieve diagnosis');
  }
}
```

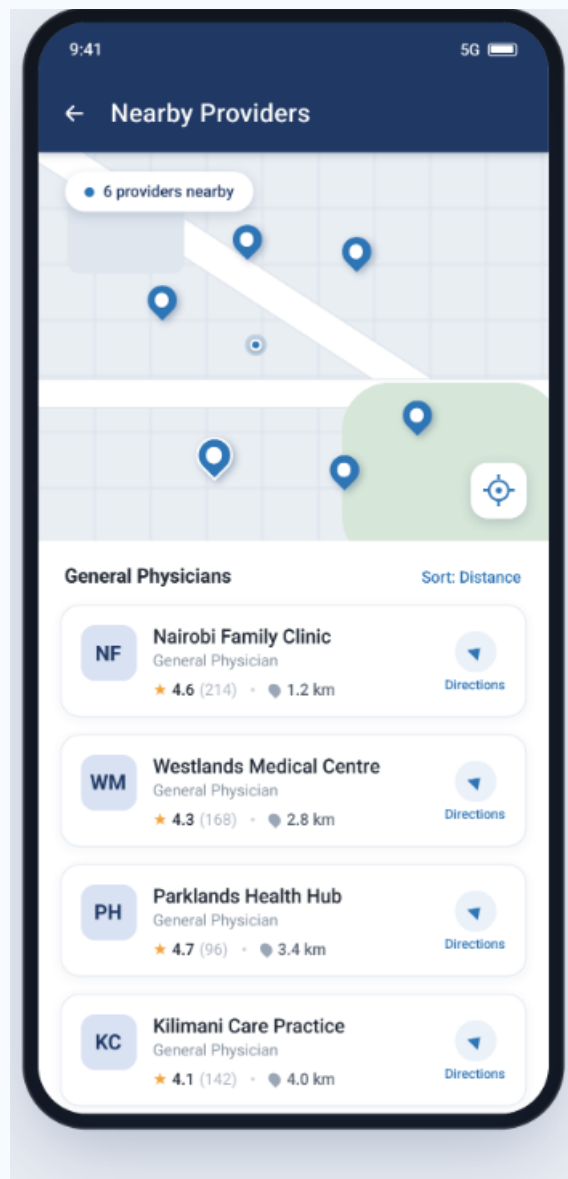
5.2 Screenshots of Interfaces and Outputs

The current alpha build implements the symptom-input screen and the results screen.



.1: Symptom input screen

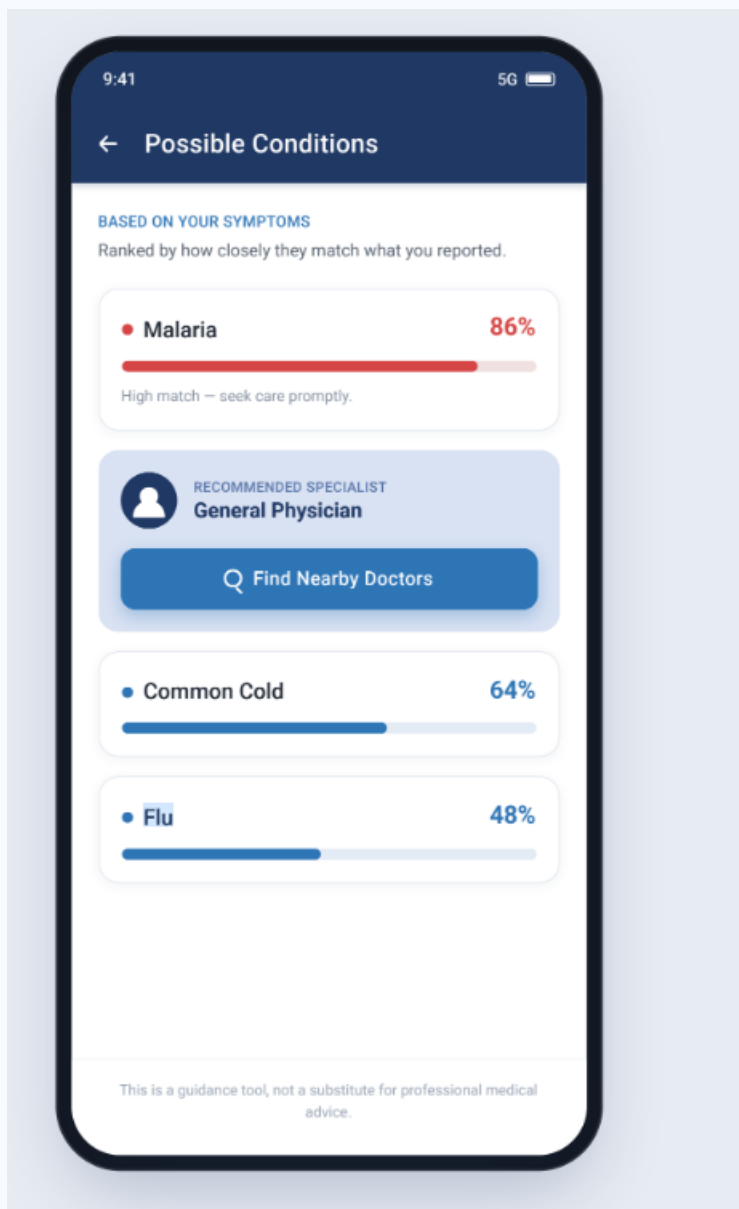
Figure 5.1: Symptom input screen.



5.2:

Ranked results and recommended specialist screen

Figure 5.2: Diagnosis results screen.



5.3:

*Nearby providers / map screen (in progress)**Figure 5.3: Nearby providers screen.*

5.3 Explanation of Key Modules and Algorithms

5.3.1 Knowledge Base Module

The knowledge base stores conditions as weighted collections of symptoms. Keeping it in a structured, declarative format (JSON / Firestore documents) means new conditions and symptoms can be added without modifying the engine code, directly satisfying the maintainability requirement from Section 3.3.2.

5.3.2 Diagnostic Engine and Scoring Algorithm

For each candidate condition, the engine sums the weights of the symptoms the user reported and divides by the total possible weight for that condition, producing a normalised confidence score between 0 and 1. Conditions scoring above a configurable threshold are returned, sorted in descending order of confidence. Because every score traces directly back to specific weighted symptoms, the reasoning is fully explainable a deliberate design choice for a safety-sensitive domain.

5.3.3 Specialist Recommendation Module

Each condition is annotated with the appropriate specialist type. Once the top-ranked condition is identified, its associated specialist is returned to the user, translating an abstract condition into a concrete, actionable next step.

5.3.4 Geolocation and Provider Module

Using the device's coordinates and the recommended specialist type, the provider service queries the Google Maps Platform for nearby facilities, ranks them (currently by rating), and returns a shortlist. Caching and graceful handling of denied location permissions are being implemented to manage API cost and respect user privacy.

5.3.5 Safety and Disclaimer Handling

Every diagnosis response carries an explicit disclaimer that the application is a guidance tool and not a substitute for professional medical advice, and high-severity conditions are flagged to encourage prompt professional consultation.

Chapter Six: Conclusions and Recommendations

6.1 Summary of Findings

The first half of the project has confirmed both the need for, and the feasibility of, an accessible, location-aware symptom-checking application. The literature review and comparative analysis established a clear gap: existing tools rarely combine explainable symptom analysis, specialist routing, and localised provider recommendation in a form suited to low-connectivity, developing-region contexts.

On the technical side, the requirements, system architecture, and data models have been finalised, and a functional prototype of the rule-based diagnostic engine has been implemented and tested against sample symptom sets. The weighted symptom–disease approach has proven both effective and transparent, producing ranked, explainable results and mapping them to appropriate specialists. The initial mobile interface screens for symptom input and results have also been built. The project is on schedule relative to the ten-week timeline, with the geolocation module and full integration representing the main remaining work.

6.2 Recommendations for Future Work

27. Complete and harden the geolocation-based provider recommendation module, including result caching and robust handling of denied location permissions.
28. Expand and clinically validate the symptom–disease knowledge base in consultation with medical references and, where possible, qualified practitioners, to improve coverage and accuracy.
29. Conduct structured user-acceptance testing with users of varying digital-literacy levels to refine usability.
30. Add multilingual support and offline-capable operation to further serve low-connectivity communities.
31. Extend the platform to iOS once the Android build is stable, leveraging the cross-platform architecture already in place.
32. Investigate, as a longer-term enhancement, a hybrid approach that augments the transparent rule base with data-driven refinement while preserving explainability.

6.3 References

References are formatted in APA 7th edition. It is recommended to manage these using Mendeley or Zotero and expand the list as the project progresses.

Google Developers. (2024). Maps Platform documentation.
<https://developers.google.com/maps/documentation>

Semigran, H. L., Linder, J. A., Gidengil, C., & Mehrotra, A. (2015). Evaluation of symptom checkers for self-diagnosis and triage: Audit study. *BMJ*, 351, h3480.
<https://doi.org/10.1136/bmj.h3480>

World Health Organization. (2023). Primary health care. <https://www.who.int/health-topics/primary-health-care>

6.4 Appendices

The following supplementary material should be attached as appendices:

- Appendix A: Full symptom–disease knowledge base (extended rule listing).
- Appendix B: UI/UX wireframes produced during the design phase.
- Appendix C: Extended code listings from the GitHub repository.
- Appendix D: Test cases and sample outputs from the diagnostic engine.